Pravara Rural Education Society's

PRAVARA RURAL ENGINEERING COLLEGE

A/P: -Loni 413 736, Tal- Rahata, Dist - Ahmednagar,

Maharashtra

DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

LABORATORY JOURNAL

FOR

MICROCONTROLLER AND APPLICATIONS (2024 COURSE)

TE - ELECTRONICS AND COMPUTER ENGINEERING

NAME: _____

ROLL NO.: _____ **EXAM SEAT NO.:** _____

Institute Vision Mission

Vision

To provide quality technical education to foster youth with skills and values to make them capable of delivering significant contributions in local to global development.

Mission

M1: Providing a conducive environment for effective teaching-learning, research, and innovation.

M2: Establishing an ecosystem to nurture state-of-the art skill sets through training, collaborating, and mentoring.

M3: Ensuring a culture of continuous improvement, effective communication, and transparency among all stakeholders.

M4: Promoting ethical and moral values to develop responsible and socially conscious citizens.

Electronics and Computer Engineering

Vision

Promote effective teaching learning in the fields of electronics and computer engineering to produce nationally, globally competitive engineers that are needed to meet the evolving demands of industry and society.

Mission

M1- To impart the latest multidisciplinary technical knowledge in the field of Electronics and Computer Engineering.

M2- To promote innovative ideas for solving multidisciplinary engineering problems and provide opportunity for creating talent in students to take up challenges for the benefits of society and industry.

M3- To promote collaboration between industry and academia for the benefit of students' overall development.

Program Educational Outcomes (PEOs)

1. Through effective teaching learning environment, the graduates will achieve a high level of technical competence in Electronics and Computer Engineering, so that they are able to develop solutions to the problems.
2. Ability to work effectively in a multidisciplinary environment individually or in team at a global and societal level.
3. Graduates will compete on a national, global platform to pursue their career in Electronics Engineering as well as in Computer Engineering and allied disciplines.

Program Specific Outcomes

PSO1: Ability to identify, analyze, design and develop applications in Electronics as well as in Computer field by using appropriate modern software, tools and techniques.

PSO2: Demonstrate the knowledge of modern software and techniques of electronics and computer engineering for the development of innovative applications needed at a global and societal level.

Program Outcomes (POs)

The Electronics and Computer Engineering graduate shall demonstrate.

PO1: **Engineering Knowledge:** Apply the knowledge of mathematics, natural science, computing, and engineering fundamentals and an engineering specialization to the solution of complex engineering problems.

PO2: **Problem Analysis:** Identify, formulate, and analyse complex engineering problems, reaching substantiated conclusions with consideration for the holistic nature of the problem.

PO3: **Design/Development of Solutions:** Design creative solutions for complex engineering problems and design systems, components, or processes to meet identified needs, with consideration for public health and safety, and cultural, societal, and environmental factors. Sustainability is intrinsic to design.

PO4: **Conduct Investigations of Complex Problems:** Conduct investigations of complex engineering problems using research-based knowledge and research methods,

including design of experiments, analysis and interpretation of data, and synthesis of information to provide valid conclusions.

PO5: **Engineering Tool Usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools, including prediction and modelling, while recognising their limitations, to solve complex engineering problems.

PO6: **The Engineer and the World:** Analyse and evaluate societal and environmental aspects while solving complex engineering problems, considering their impact on sustainability with reference to economy, health, safety, legal frameworks, culture, and the environment.

PO7: **Ethics:** Apply ethical principles and commit to professional ethics, human values, diversity, and inclusion, and adhere to relevant national and international laws.

PO8: **Individual and Collaborative Team Work:** Function effectively as an individual, and as a member or leader in diverse and multidisciplinary teams.

PO9: **Communication:** Communicate effectively and inclusively within the engineering community and with society at large, including comprehending and writing effective reports and design documentation, and making effective presentations, while considering cultural, language, and learning differences.

PO10: **Project Management and Finance:** Apply knowledge and understanding of engineering management principles and economic decision-making to one's own work, as a member and leader in a team, to manage projects in multidisciplinary environments.

PO11: **Life-Long Learning:** Recognize the need for, and have the preparation and ability for, independent and lifelong learning, adaptability to new and emerging technologies, and critical thinking in the broadest context of technological change.

Course Objectives:

- To develop hands-on skills in interfacing peripheral devices such as LED, LCD, DAC, and motors with 8051 and RL78 microcontrollers using embedded C programming.
- To understand and apply timer, interrupt, and multiplexing techniques for controlling displays, sensors, and actuators in embedded system applications.
- To develop and implement microcontroller-based circuits for real-time signal generation, motor control, and sensor-based monitoring using standard driver ICs and modules.
- To familiarize students with industry-relevant development tools and hardware platforms including 8051 and RL78 for building practical embedded applications.

Course outcomes:

After completion of this course students are Able to

CO1: Apply [L3: Application]: Apply knowledge to develop Interfacing and program LED banks, seven-segment displays, LCD, and RGB LEDs with 8051 and RL78 microcontrollers using timer and interrupt operations.

CO2: Develop [L3: Application]: Develop embedded C programs for DAC waveform generation, stepper motor control, and DC motor speed/direction control using 8051 microcontrollers

CO3: Apply [L3: Application]: Interface IR obstacle sensor, LDR, and LM35 temperature sensor with RL78 microcontroller for real-time detection and monitoring applications.

CO4: Develop [L3: Application]: Apply ADC operation, echo pulse timing, and digital output control techniques using RL78 microcontroller for sensor-based embedded system development.

PRAVARA RURAL ENGINEERING COLLEGE
LONI

CERTIFICATE

This is to certify that **Mr./Ms.**_____a student of **TE Electronics and Computer Engineering** has performed the above mentioned 9 **(Nine)** experiments (Counter signed in the index sheet) in the **IoT and Embedded System Lab** of the **Pravara Rural Engineering College, Loni.**

STAFF INHARGE
DATE : / /2026

HEAD OF DEPT
Electronics and Computer Engg.

PRAVARA RURAL EDUCATION SOCIETY
PRAVARA RURAL ENGINEERING COLLEGE
LONI

Department of Electronics and Computer Engineering

Subject: Microcontroller and Applications

T.E. (ELECTRONICS AND COMPUTERCENGINEERING) (2024 Course)

INDEX

Expt. No	Date	Title	Page No.	Remark	Signature
1.		Interfacing LED bank to 8051 microcontroller using timer with interrupt.			
2.		To interface a Seven Segment Display with the 8051 microcontrollers for displaying numeric digits and counter values.			
3.		To interface a DAC with the 8051 microcontrollers for generating different analog waveforms such as square, triangular, and sine waves.			
4		To interface a 16×2 LCD display module with the 8051 microcontroller and display alphanumeric characters, messages			
5		To interface a stepper motor with the 8051 microcontrollers for controlling rotational direction, step angle, and speed operation.			
6		To interface a DC motor with the 8051-microcontroller using motor driver IC for controlling motor speed and direction in embedded control applications.			
7		To interface an LED with the RL78 microcontroller and write a program for LED blinking for understanding digital output port operations and delay generation			
8		To interface an IR obstacle detection sensor with the RL78 microcontroller for detecting nearby objects and controlling indication devices based on sensor output.			
9		To interface a 16×2 LCD display module with the RL78 microcontroller and display alphanumeric characters, messages, and system status information			

Signature of Subject Teacher **Head of Department**

Date: / /2026

EXPERIMENT NO-1

Aim: - To interface an LED bank with the 8051-microcontroller using timer and interrupt operation for sequential LED blinking and timing control applications.

Apparatus: - 89c51 microcontroller kit, connecting wires.

Software: -Keil version 5 software, SSTFLASHFLEX etc.

Theory: -

LEDs are human oriented output peripheral. It is used to display result or operand.

To drive a LED, there are two methods.

Method 1:

Connect the cathode of LED to ground. Connect the anode of LED to port pin of 8051, through a resistor as shown in fig A.

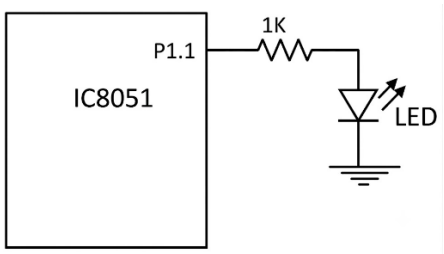


Fig.1: LED driven by 8051 port pin (method 1)

This method requires 8051 to source a huge amount of current required by the LED i.e. around 20mA.But 8051is not capable of sourcing a current more than 2mA.this will make the LED glow very dim.

Method 2:

Connect the anode of LED to Vcc through resistor. Connect the cathode of LED to port 8051 as shown in fig B. This method requires 8051 to sink a huge current required by LED i.e. 20mA.8051 can sink huge currents and hence it makes LED glow brighter. Hence we will always use method 2, to interface LED.

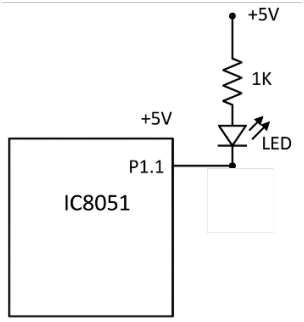


Fig 2: LED driven by 8051 port pin (Method 2)

LED flashing:

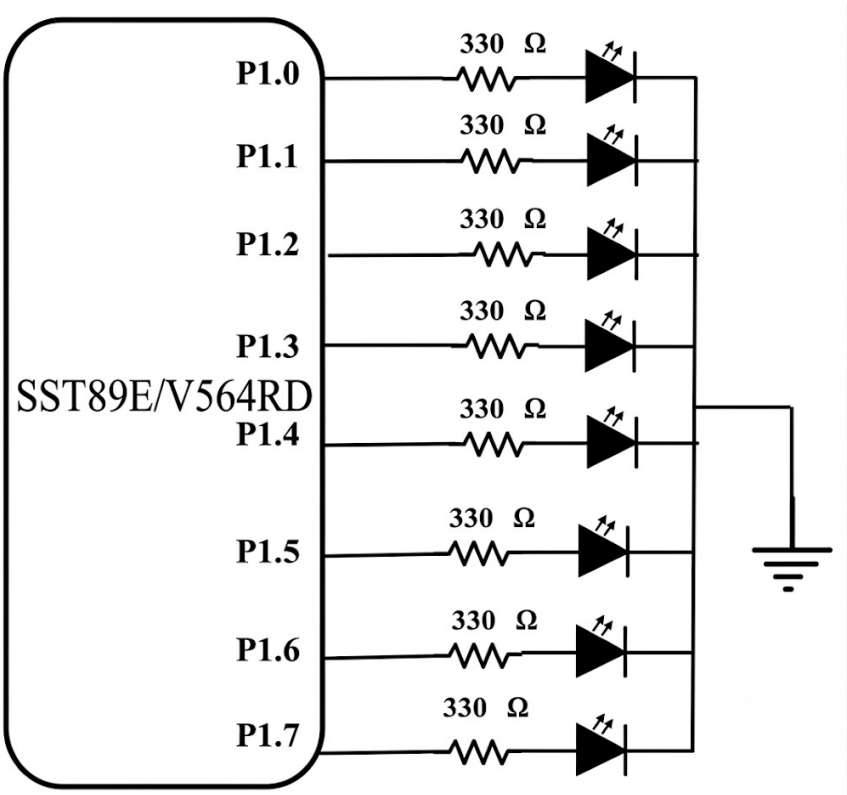
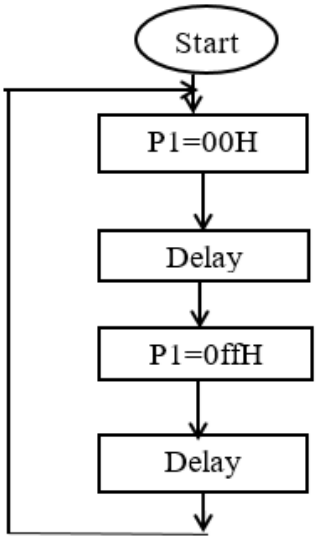


Fig 3: LED interfacing using IC 89x51

Figure 3 shows interfacing of eight LEDs with port 1 of 8051 with current limiting register of 330 Ω.

Flowchart for LED Flashing



Program for Single LED Blinking-

```
#include<reg52.h>          // special function register declarations

sbit LED = P2^3;           // Defining LED pin

void Delay(void);          // Function prototype declaration

void main (void)
{
   while(1)               // infinite loop
   {
      LED = 0;            // LED ON
      Delay();
      LED = 1;            // LED OFF
      Delay();
   }
}

void Delay(void)
{
   int j;
   int i;
   for(i=0;i<10;i++)
   {
      for(j=0;j<10000;j++)
      {
```

```
        }
    }
        }
```

Program for LED Flashing-

```
#include <REG51.h>

void delaymS(unsigned long x)
{
        while(x > 0 )
        {
                unsigned int i=0;
                for( i=0; i<500;i++);           //approximately 1mS
                x--;
        }
}

void main()
{
        while(1)
        {
                P2 = 0x00;              //Turn off all LEDs
                delaymS(100);// delay time between the blinking
                P2 = 0xff ;     //Turn on all LEDs
                delaymS(100);// delay time between the blinking
        }
}
```

Procedure to use the Keil software:-

Step 1] Open Keil 5 Software
Step 2] Close old Project
Step 3] Create new Project
Step 4] Save file with extension .c
Step 5] Search

- Select **SST89E564RD**
- Click Ok

Step 6] Right Click on Target-1

- Click on Output
- Click on Create Hex file
- Click Ok

Step 7] Right Click on Source

- Click on Add Existing File
- Add Created File (ex.c)

Step 8] Double click on Added file

Step 9] Type Program

- Save
- Rebuild

Step 10] Right click on my Computer

- Click on Properties
- Click on Device Manager
- Click on Port(ex.port5) *//Remind Port No*.

Step 11] Right click on SSTFLASHFLEX Software

- Click on Run as Administrator
- Click Yes

Step 12] Click on Option

- Check whether Device name and Chip Type is same or Select SST89E/V564RD
- Select COMPORT (ex.port5)
- Click Ok

Step 13] Click RESET button on Kit

Step 14] Click on Dnload/Load

- Click on Browse File
- Select Hex file we created
- Click Ok

Conclusion: -

The LEDs are successfully interfaced with the 89C51 microcontroller. LED flashing is successfully observed by controlling the output port.

EXPERIMENT NO-2

AIM: To interface a Seven Segment Display with the 8051 microcontrollers for displaying numeric digits and counter values.

OBJECTIVE: To achieve interfacing of seven segments LED display and generate counting from 0 to 9 with fixed time delay.

Software Requirement: Keil version 5 software, SSTFLASHFLEX etc.

Hardware Requirement: Personal Computer, 8051 board, +5 V power supply,

THEORY: -

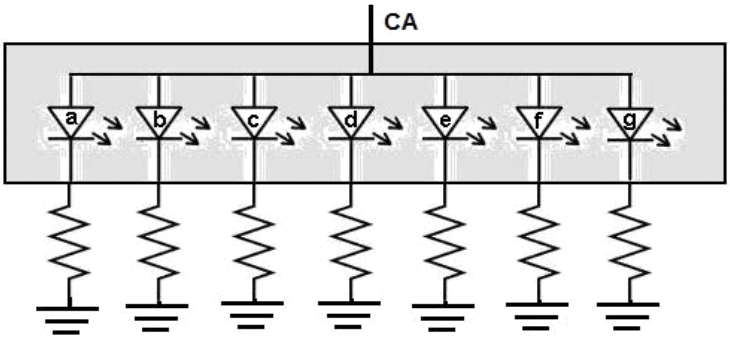
Seven segment LED displays are often found in clock radios, VCRs, microwave ovens, toys and many other household items. They are primarily used to display decimal numbers but they can also display a few alphabets and other characters. This experiment describes interfacing a seven segment LED display to a 89V51RD2 microcontroller. A seven segment LED display is an special arrangement of 7 LED elements to form a rectangular shape using two vertical segments on each side with one horizontal segment on the top, middle, and bottom. By individually turning the segments on or off, numbers from 0 to 9 and some letters can be displayed. Seven segment displays sometime also have an eighth segment to display the decimal point. Therefore, a seven-segment display will require seven outputs from the microcontroller to display a number, and one more output if the decimal point is to be displayed too.

The segments are marked with non-capital letters: a, b, c, d, e, f, g and dp, where dp is the decimal point. The 8 LEDs inside the display can be arranged with a common cathode or common anode configuration. With a common cathode display, the cathodes of all the segment LEDs are tied together and this common point must be connected to the ground. A required LED segment is then turned on by applying a logic 1 to its anode. In common anode displays, all the anodes are tied together and the common anode is connected to the supply voltage Vcc. Individual segments are turned on by applying logic 0 to their cathodes.

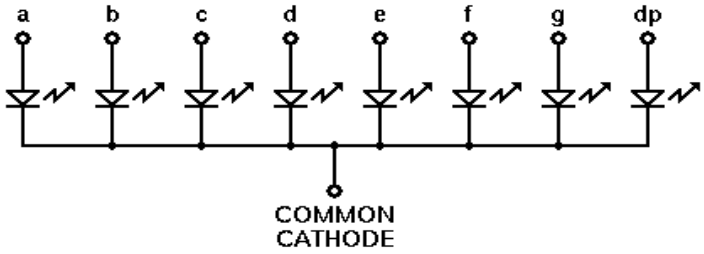
When more than one seven segment display is used, a multiplexing technique is used to minimize the required number of microcontroller pins.

There are two types of seven segment display:

1. **Common Anode**



2. **Common Cathode**

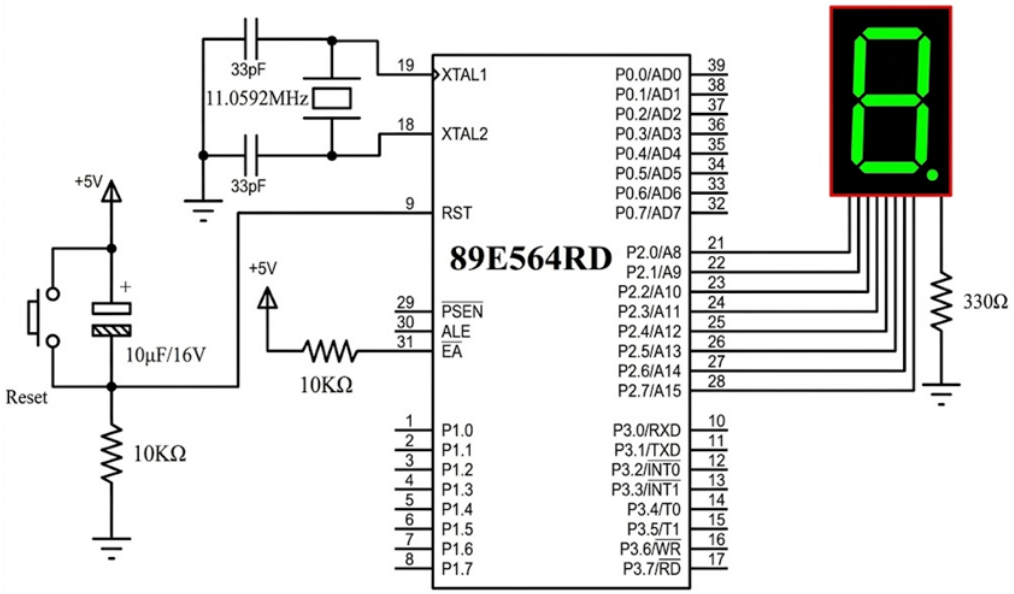


Digit Drive Pattern:

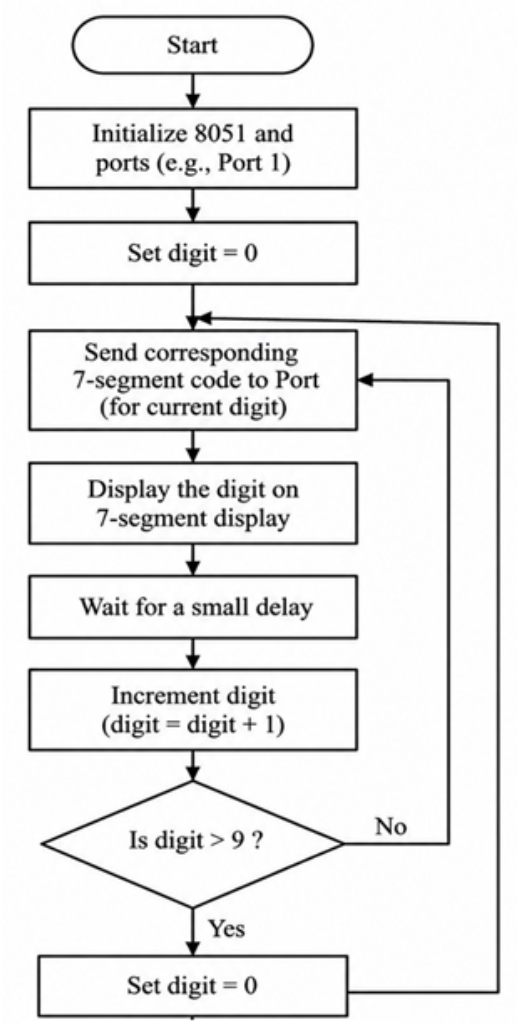
Digit drive pattern of a seven segment LED display is simply the different logic combinations of its terminals 'a' to 'h' in order to display different digits and characters. The common digit drive patterns (0 to 9) of a seven segment display are shown in the table below.

DIGIT	DP	G	F	E	D	C	B	A	HEX Value
0	0	0	1	1	1	1	1	1	0x3f
1	0	0	0	0	0	1	1	0	0x06
2	0	1	0	1	1	0	1	1	0x5b
3	0	1	0	0	1	1	1	1	0x4f
4	0	1	1	0	0	1	1	0	0x66
5	0	1	1	0	1	1	0	1	0x6d
6	0	1	1	1	1	1	0	1	0x7d
7	0	0	0	0	0	1	1	1	0x07
8	0	1	1	1	1	1	1	1	0x7f
9	0	1	1	0	0	1	1	1	0x67

Interfacing Diagram


33pF
11.0592MHz
33pF
+5V
+5V
10µF/16V
Reset
10KΩ
10KΩ
330Ω
89E564RD
XTAL1
XTAL2
RST
PSEN
ALE
EA
P0.0/AD0
P0.1/AD1
P0.2/AD2
P0.3/AD3
P0.4/AD4
P0.5/AD5
P0.6/AD6
P0.7/AD7
P2.0/A8
P2.1/A9
P2.2/A10
P2.3/A11
P2.4/A12
P2.5/A13
P2.6/A14
P2.7/A15
P1.0
P1.1
P1.2
P1.3
P1.4
P1.5
P1.6
P1.7
P3.0/RXD
P3.1/TXD
P3.2/INT0
P3.3/INT1
P3.4/T0
P3.5/T1
P3.6/WR
P3.7/RD

Flowchart:


Start
Initialize 8051 and ports (e.g., Port 1)
Set digit = 0
Send corresponding 7-segment code to Port (for current digit)
Display the digit on 7-segment display
Wait for a small delay
Increment digit (digit = digit + 1)
Is digit > 9 ?
No
Yes
Set digit = 0

Program :

```
#include <REG51.h>
#define SEGPORT  P2       // Port to which 7-SEG is connect
sbit SS1 = P3^6;          // segment select 1
sbit SS2 = P3^7;          // segment select 2

/* Bit pattern for Dispaying Numerals, starting from 0 to 9 */
const unsigned char LookupTable[] = {0x40,0xF9,0x24,0x30,0x19,0x12,0x02,0x78,0x00,0x10};
// common cathode
// const unsigned char
LookupTable[10]={0xF6,0x90,0xE5,0xB5,0x93,0x37,0x77,0x94,0xF7,0xB7}; // common anode


void delaymS(unsigned long x)  // Function to generate a dely of approximate 1 millisecond
{
   unsigned int i,j;

   for(i=0;i<x;i++)
      for(j=0;j<1000;j++);
}


void main()
{
   int i=0;
   unsigned char x=1;

   SEGPORT = 0x00;
   SS1 = 1;
   SS2 = 1;

   while(1)
   {
      for(i=0;i<10;i++)
      {
         SEGPORT = ~LookupTable[i];  // get the pattern from the LookUp Table
         delaymS(100);               // delay for the next number
      }
   }
}
```

Procedure to use the Keil software: -

Step 1] Open Kill 5 Software

Step 2] Close old Project

Step 3] Create new Project

Step 4] Save file with extension **.c**

Step 5] Search

- Select SST**89E564RD**
- Click Ok

Step 6] Right Click on Target-1

- Click on Output
- Click on Create Hex file
- Click Ok

Step 7] Right Click on Source

- Click on Add Existing File
- Add Created File (ex.c)

Step 8] Double click on Added file

Step 9] Type Program

- Save
- Rebuild

Step 10] Right click on my Computer

- Click on Properties
- Click on Device Manager
- Click on Port(ex.port5) *//Remind Port No*.

Step 11] Right click on **SSTFLASHFLEX Software**

- Click on Run as Administrator
- Click Yes

Step 12] Click On Option

- Checkwhether Device name and Chip Type is same or Select **SST89E/V564RD**
-

- Select COMPORT (ex.port5)
- Click Ok

Step 13] Click **RESET** button on Kit

Step 14] Click on Dnload/Load

- Click on Browse File
- Select **Hex file** we created
- Click Ok

Conclusion: The common cathode seven-segment LED display is successfully interfaced with the 89C51 microcontroller. Counting from 0 to 9 is successfully observed on the display.

EXPERIMENT NO-3

TITLE: -WAVEFORM GENERATION USING DAC

Aim: - To interface a DAC with the 8051 microcontrollers for generating different analog waveforms such as square, triangular, and sine waves.

Apparatus: - 89v51 microcontroller kit, connecting wires.

Software Requirement: Keil version 5 software, SSTFLASHFLEX etc.

Hardware Requirement: Personal Computer, 8051 board, +5 V power supply,

Theory: -

DAC 0808

The DAC 0808 is an 8-bit current output monolithic DAC manufactured by the National semiconductor corporation. It is a 16 - pin IC available in dual in line DIP plastic package. The analog output is available in the form of current Io. That means Io is proportional to the 8-bit digital input. The important features of DAC 0808 are as follows:

Features of DAC 0808:

Fast settling time	150 nsec. typically
Power supply voltage range	+-4.5 mW at +-5V
Low power consumption	33 mW at +-5V.
High speed multiplying input slew rate	8 mA/ micro sec.

Interfaces directly with TTL, DTL, and CMOS logic levels.

Pin Configuration and Functional Block Diagram

The pin configuration and functional block diagram of DAC 0808 are as shown in Fig(a) and Fig(b) respectively.

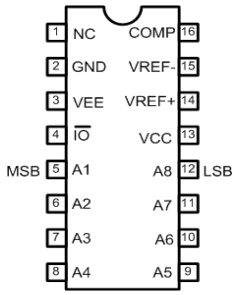


Fig1: Pin configuration of DAC 0808

- The internal block diagram shows that DAC 0808 consists of R-2R ladder along with

current switches and reference current amplifier.

- A1 to A8 are the 8-digital input lines with A1 as the most significant bit and A8 as the least significant bit.
- The analog output is available in the form of current Io, therefore we need to use an external current to voltage converter if the analog output in the form of voltage is required.
- DAC 0808 requires a dual polarity (+-) supply voltage, typically +- 15V, for its operation. The reference voltage can be either positive or negative.
- An external reference voltage should be applied to either Vref(+) or Vref(-) depending on the polarity of reference voltage.

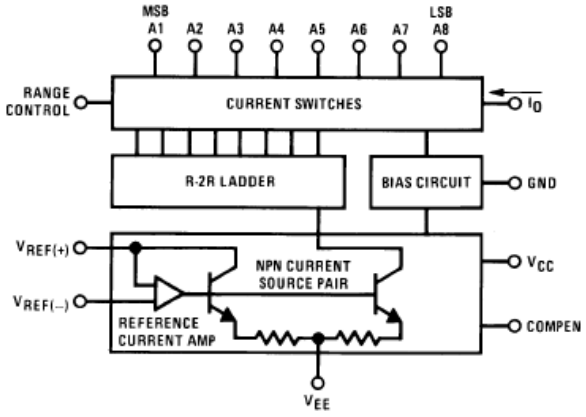


Fig2: Functional block diagram of DAC 0808

Interfacing DAC to 0808

Fig(c) shows the interfacing of DAC 0808 to 8051.

- The output of DAC is a current which is converted into voltage using op-amp based current-to-voltage (I-V) converter.

- The analog output current Iout of the DAC depends on the Iref flowing into the Vref terminal and the status of the D0 - D7 bits. It is expressed as,

Iout=Iref((D7/2) + (D6/4) + (D5/8) + (D4/16) + (D3/32) +(D2/64) +(D1/128) +(D0/256))

Where, D7=MSB

Iref depends on Vref voltage and the resistors 1Kohm and 1.5Kohm connected.

Iref=Vref/(1K+1.5K) =5V/2.5Kohm=2mA

(a) If D0-D7= FFH then

Iout=2mA*((1/2) +(1/4) +(1/8) +(1/16) +(1/32)+(1/64)+(1/128)+(1/256))

= 2mA*(255/256)

=1.99mA

Vout=1.99mA*5Kohm

=9.96V

(b) If D0-D7=80H then

Iout=2mA*((1/2) +(0/4) +(0/8) +(0/16)+(0/32)+(0/64)+(0/128)+(0/256))

=1mA

Vout=1mA*5Kohm=5V

(c) If D7-D0=00H then Iout=0, Vout=0V

DAC is commonly used in waveform generation.

Interfacing Diagram:

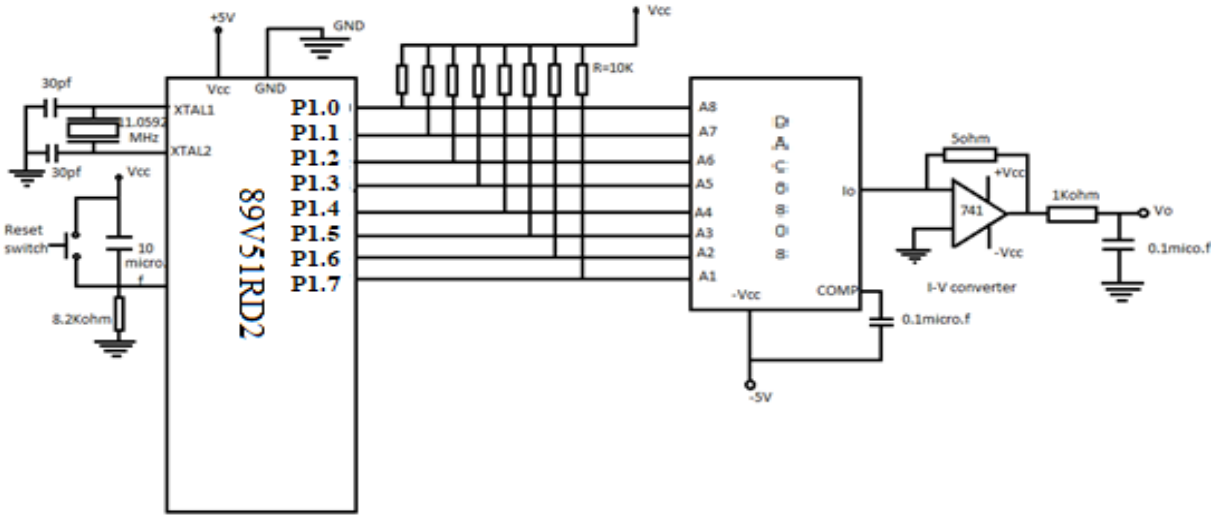
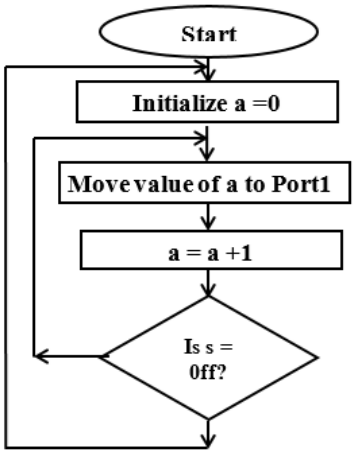
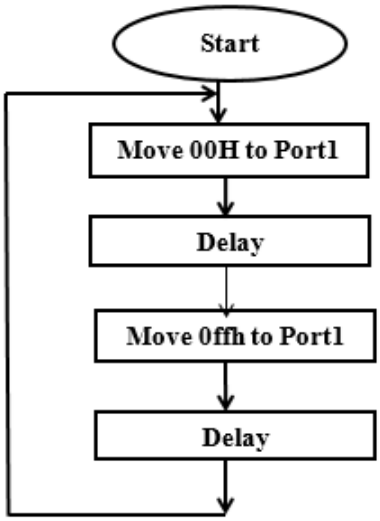


Fig 3: Interfacing of DAC 0808 to 89v51RD2

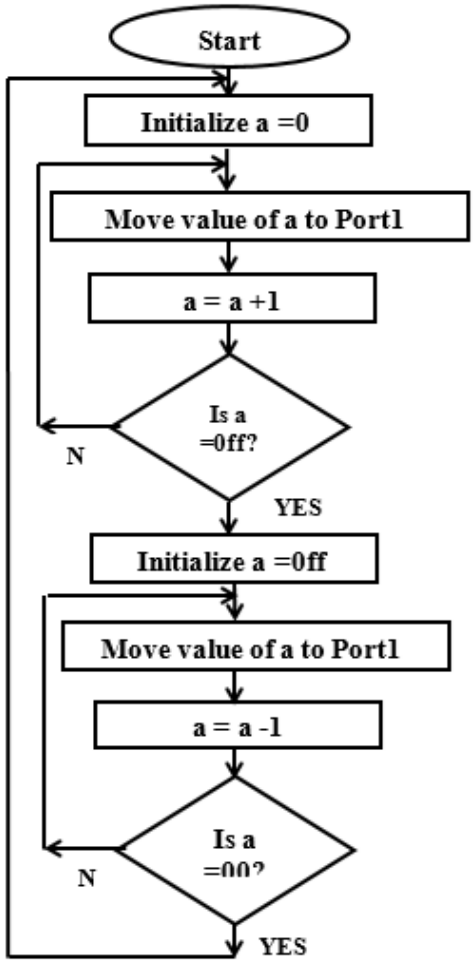
Flow chart for Ramp Wave



Flow chart for Square Wave



Flow chart for Triangular Wave



1. Program for Ramp wave-

```
#include <REG51.h>

void main(void)
{
   unsigned int a;

   while (1)
   {
      for (a = 0x00; a < 0xff; a++)
      {
         P1 = a;
      }
   }
}
```

2. Program for Triangular Wave

```
#include <REG51.h>

void main(void)
{
   unsigned int a;

   while (1)
   {
      // rising ramp edge
      for (a = 0x00; a < 0xFF; a++)
      {
         P1 = a;
      }

      // falling ramp edge
      for (a = 0xFF; a > 0; a--)
      {
         P1 = a;
      }
   }
}
```

3. Program for Square Wave

```
#include <REG51.h>

void delay_time(unsigned int time)
{
   unsigned int i, j;

   for (i = time; i > 0; i--)
   {
      for (j = 0; j < 2000; j++);
   }
}

void send_dac(unsigned int dat)
{
   P1 = dat;
}

void main(void)
{
   unsigned int delay = 0xFF;
```

```
    unsigned char state = 0xFF;

    send_dac(0x7F);

    while (1)
    {
       send_dac(state);
       delay_time(delay);

       send_dac(~state);
       delay_time(delay);
    }
}
```

4.Program for Sinewave :-

```
#include<reg51.h>

void main()
{
   unsigned char WAVEVALUE[12]= {128, 192, 238, 255, 238, 192, 128, 64, 17, 0, 17, 64};
   unsigned char x;

   while(1)
   {
      for(x=0;x<12;x++)
      {
         P1= WAVEVALUE[x];
      }
   }
}
```

Procedure to use the Keil software:-

Step 1] Open Keil 5 Software
Step 2] Close old Project
Step 3] Create new Project
Step 4] Save file with extension .c
Step 5] Search

- Select **SST89E564RD**
- Click Ok

Step 6] Right Click on Target-1

- Click on Output
- Click on Create Hex file
- Click Ok

Step 7] Right Click on Source

- Click on Add Existing File
- Add Created File (ex.c)

Step 8] Double click on Added file

Step 9] Type Program

- Save
- Rebuild

Step 10] Right click on my Computer

- Click on Properties
- Click on Device Manager
- Click on Port(ex.port5) *//Remind Port No*.

Step 11] Right click on SSTFLASHFLEX Software

- Click on Run as Administrator
- Click Yes

Step 12] Click on Option

- Check whether Device name and Chip Type is same or Select SST89E/V564RD
- Select COMPORT (ex.port5)
- Click Ok

Step 13] Click RESET button on Kit

Step 14] Click on Dnload/Load

- Click on Browse File
- Select Hex file we created
- Click Ok

Conclusion: -

The 89C51 microcontroller is successfully interfaced with the DAC logic board. Square, ramp, and sine waveforms are successfully generated and observed.

EXPERIMENT NO: 4

TITLE: - LCD INTERFACING

Aim: - To interface a 16×2 LCD display module with the 8051 microcontroller and display alphanumeric characters, messages

Apparatus: - 89c51 microcontroller kit, connecting wires.

Software: -Keil software, SST Flash etc.

Theory: -

Interfacing of liquid crystal Display (LCD)

LCD displays are widely used because of its low current consumption as compared to SSD. Also that LCD CAN BE used to display any character as it uses to display any character as it uses a 5*6 dot matrix to display.

LCD Pin Description

Table: Pin description of LCD

PINS	SYMBOL	FUNCTION
1	VSS	Ground
2	VCC	+5v supply
3	VEE	Power supply to control contrast
4	RS	Should be ‘0’ for instruction and ‘1’ for data
5	R/W	Should be ‘0’ to write and ‘1’ to read
6	E	Enable display logic
7	D0	Data bus bit0
8	D1	Data bus bit1
9	D2	Data bus bit2
10	D3	Data bus bit3
11	D4	Data bus bit4
12	D5	Data bus bit5
13	D6	Data bus bit6
14	D7	Data bus bit7 (also used as busy pin)

(A)RS: Registers Select

- There are two register of LCD viz. Instruction command code register and data register. The Rs pin is used to select one of these register.
- IF RS=0 the instruction command code register is selected and if RS =1 The data register is selected allowing user to send data to be displayed on the LCD.

(B) R/w: Read/Write

- R/W pin is used to read or write to LCD.
- IF R/W=0 the user can write information to the LCD and if R/W=1 the user can read information.

(C) Enable: E (it can be also be denoted by EN)

- The enable pin E. is used to latch the data into the data or command register. When data is supplied a high-to low (negative edge) is required for LCD to latch the data.

(D)D0-D7 OR DBO-DB7

- The data pins DO-D7 are used to send to send information to the LCD or read the contents of LCD internal register.
- To display letter and number we send ASCIT code for letter A-Z, a-z and numbers 0-9 while making RS=1.
- The ASCII code that is to be displayed is of 8 bits. it is send to the LCD in either nibbles or bytes i.e.4 or8 bits at a time.
- the two primary modes of operation to send parallel data are 4or 8 bits.
- If four-bit mode is used two nibbles of data are sent to do an 8-bit transfer. The “E” clock is used to initiate the data transfer. At least 6 I/O pins must be available for 4-bit mode.
- In 8-bit mode at least 10 I/O pins must be used. This mode is used when application needs speed.

(E) VCC, VSS and VEE

VCC-provides +5 supply

VSS-Ground

VEE-is used for controlling LCD contract.

Cursor Addresses for LCDs:

16x2 LCD	**80**	**81**	**82**	**83**	**84**	**85**	**86**	**through**	**8F**
	C0	**C1**	**C2**	**C3**	**C4**	**C5**	**C6**	**through**	**CF**
20X1 LCD	**80**	**81**	**82**	**83**	**through**	**93**			
20X2 LCD	**80**	**81**	**82**	**83**	**through**	**93**			
	C0	**C1**	**C2**	**C3**	**through**	**D3**			
20X4 LCD	**80**	**81**	**82**	**83**	**through**	**93**			
	C0	**C1**	**C2**	**C3**	**through**	**D3**			
	94	**95**	**96**	**97**	**through**	**A7**			
	D4	**D5**	**D6**	**D7**	**through**	**E7**			
40X2LCD	**80**	**81**	**82**	**83**	**through**	**A7**			
	C0	**C1**	**C2**	**C3**	**through**	**E7**			
All data in hex									

LCD Command:

- The list of commands that can be given to the LCD are as listed in the table.

HEX COMMAND	FUNCTION
0x01	Clear display
0x02	Return cursor to home
0x04	Decrement cursor (i.e. shift cursor left)
0x06	Increment cursor (i.e. shift cursor right)
0x05	Shift display right
0x07	Shift display left
0x08	Display off, cursor off
0x0A	Display off, cursor on
0x0C	Display on, cursor off
0x0E	Display on, cursor on
0x0F	Display on, cursor on and blinking
0x10	Move cursor on position left
0x14	Move cursor on position right
0x18	Shift entire display left
0x1C	Shift entire display right
0x80	Move cursor to beginning of first line
0xC0	Move cursor to beginning of second line
0x38	Initialize 2 line display of 5x7 matrix

Initialization of LCD

The following algorithm is required to initialize and write data to LCD:

- Wait second after power up for display to stabilize.
- Initialize the LCD by giving the instruction 0x38 to command subroutine.
- Wait for 5 msec.
- Issue the command 0x0F to command subroutine or display on, cursor on and cursor blinking.
- Wait for 5 msec.
- Issue the command 0x01 for clearing display to command subroutine.
- Wait for 5 msec.
- Issue the command oxo6 for making LCD in increment mode i.e. cursor should increment after every character is written to command subroutine.
- Wait for 5 msec.
- Issue the command 0x80(to command subroutine), to position the cursor at 1st line 1st character.
- Issue the data character one by one giving their ASCII value using data subroutine.

Command subroutine

- Give the Upper Nibble and then Lowe Nibble data to data bus of the LCD.
- Make RS= ‘0’, TO indicate instruction.
- Make R/W= ‘0‘to indicate write.
- Make E= ‘1’
- Wait for 12usec.
- Make E= ‘0’
- Return.

Data subroutine

- Give the Upper Nibble and then Lowe Nibble data to data bus of the LCD.
- Make RS= ‘1’, to indicate data
- make R/W=’0’ indicate write
- Make E= ‘1’
- Wait for 12usec.
- Make E= ‘0’

- Return

Interfacing LCD Module with 8051

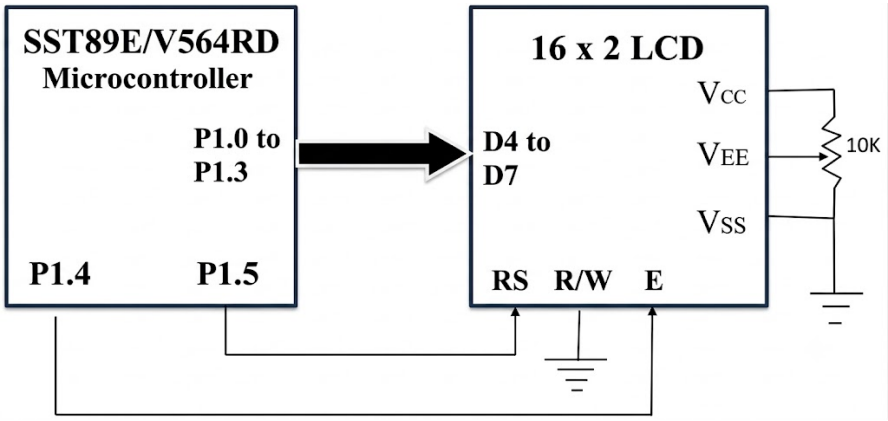
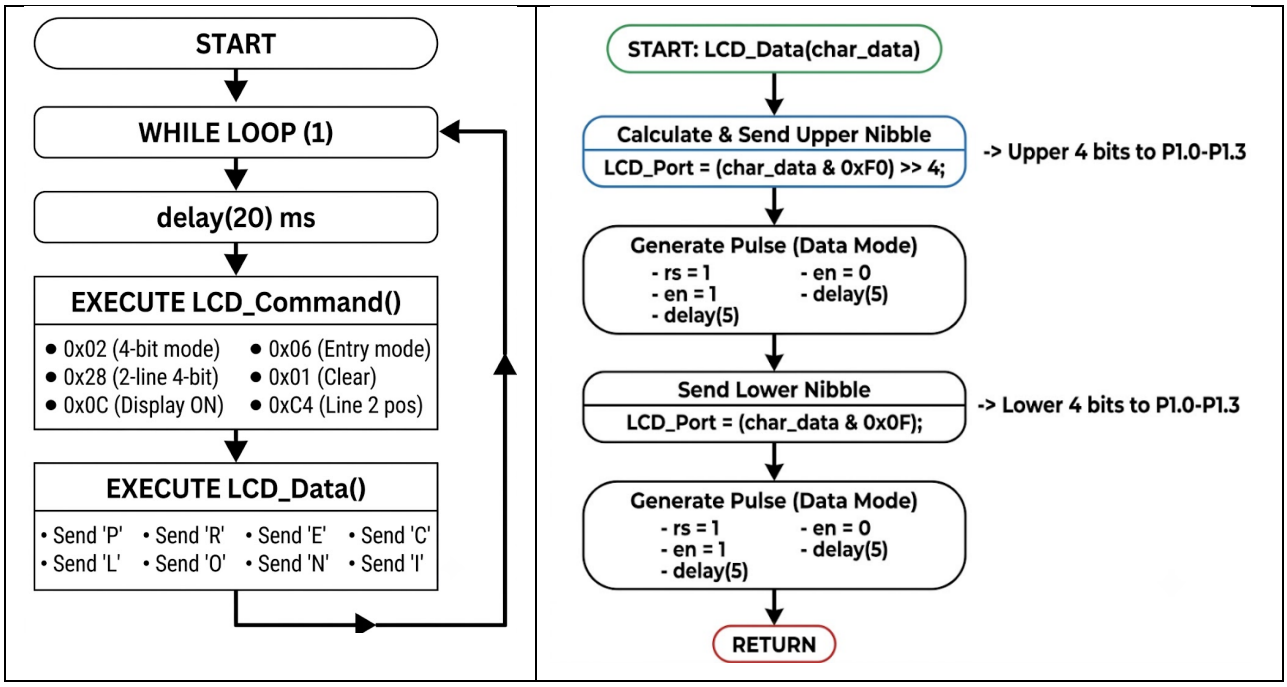
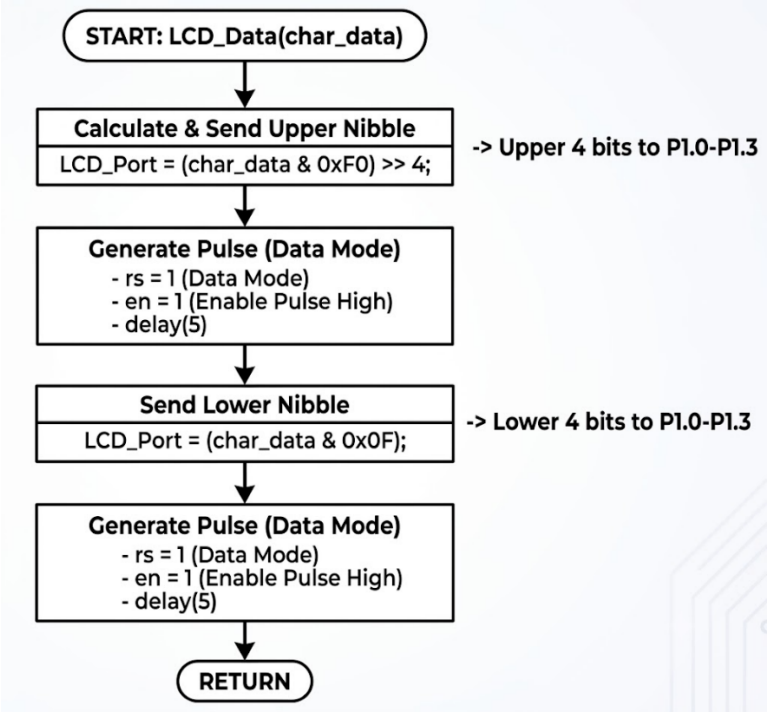


Fig: Interfacing diagram of 89x51

Flowchart: -





Program: -

```
#include <reg51.h>
#define LCD_Port P1

sbit rs = P1^5; /* Register select pin */
sbit en = P1^4; /* Enable pin */

void delay(unsigned int count)
{
   int i, j;

   for (i=0; i<count; i++)
      for (j=0; j<112;j++);
}

void LCD_Command (char cmnd) /* LCD16x2 command function */
{
   LCD_Port = (cmnd& 0xF0) >> 4; /* Send upper nibble */
   rs=0; /* Command reg. */
   en=1;
   delay(5);
   en=0;
   delay(5);

   LCD_Port = (cmnd& 0x0F); /* Send lower nibble */
   rs=0; /* Command reg. */
```

```
    en=1;
    delay(5);
    en=0;
    delay(5);
}

void LCD_Data(char char_data) /* LCD data write funcOon */
{
    LCD_Port=(char_data& 0xF0) >> 4; /* Send upper nibble */
    rs=1; /*Data reg.*/
    en=1;
    delay(5);
    en=0;
    delay(5);

    LCD_Port = (char_data& 0x0F); /* Send lower nibble */
    rs=1; /*Data reg.*/
    en=1;
    delay(5);
    en=0;
    delay(5);
}

int main()
{
    while(1)
    {
        delay(20); /* LCD Power ON IniOalizaOon Ome >15ms */
        LCD_Command (0x02); /* 4bit mode */
        delay(5);
        LCD_Command (0x28); /* IniOalizaOon of 16X2 LCD in 4bit mode */
        delay(5);
        LCD_Command (0x0C); /* Display ON Cursor OFF */
        delay(5);
        LCD_Command (0x06); /* Auto Increment cursor */
        delay(5);
        LCD_Command (0x01); /* clear display */
        delay(5);
        LCD_Command (0xC4); /* cursor at home posiOon */
        delay(5);

        LCD_Data ('P');
        delay(5);
        LCD_Data ('R');
        delay(5);
        LCD_Data ('E');
        delay(5);
        LCD_Data ('C');
```

```
        delay(5);
        LCD_Data ('L');
        delay(5);
        LCD_Data ('O');
        delay(5);
        LCD_Data ('N');
        delay(5);
        LCD_Data ('I');
        delay(5);
    }
}
```

Procedure to use the Keil software: -

Step 1] Open Keil 5 Software

Step 2] Close old Project

Step 3] Create new Project

Step 4] Save file with extension .c

Step 5] Search

- Select SST**89E564RD**
- Click Ok

Step 6] Right Click on Target-1

- Click on Output
- Click on Create Hex file
- Click Ok

Step 7] Right Click on Source

- Click on Add Existing File
- Add Created File (ex.c)

Step 8] Double click on Added file

Step 9] Type Program

- Save
- Rebuild

Step 10] Right click on my Computer

- Click on Properties
- Click on Device Manager
- Click on Port(ex.port5) *//Remind Port No.*

Step 11] Right click on SSTFLASHFLEX Software

- Click on Run as Administrator
- Click Yes

Step 12] Click on Option

- Check whether Device name and Chip Type is same or Select SST89E/V564RD
- Select COMPORT (ex.port5)
- Click Ok

Step 13] Click RESET button on Kit

Step 14] Click on Dnload/Load

- Click on Browse File
- Select Hex file we created
- Click Ok

Conclusion: -

The LCD is successfully interfaced with the 89C51 microcontroller. The required message string is successfully displayed on the LCD.

EXPERIMENT NO: 5

TITLE: - INTERFACING OF STEPPER MOTOR WITH 8051 MICROCONTROLLERS

Aim: - To interface a stepper motor with the 8051 microcontrollers for controlling rotational direction, step angle, and speed operation.

Apparatus: - 89X51 microcontroller kit, STEPPER MOTOR logic interfacing card, connecting wires.

Software: -Keil, SST Flash etc.

Theory: -

A stepper motor is a device that translates electrical pulses to mechanical movement. It is used in applications like robotics, dot-matrix printers, disk drives for position control. Stepper motors have a permanent magnet rotor surrounded by a stator.

Generally, the stepper motors have four stator windings that are paired with common center tap as shown in fig.1. Such a stepper motor is called as four phase or unipolar stepper motor. With the help of center tap the current direction in each of the two coils can be changed when a winding is grounded. This results in a polarity change of stator.

The stator is responsible for the direction of the rotation by the stator poles. The stator poles are determined by the current sent through the wire coils. As the direction of current is changed, the polarity is also changed causing the reverse motion of the stepper motor.

The stepper motor has 6 leads, 4 leads representing 4 stator windings and 2 leads for 2 commons for the center tapped leads. On application of power to the stator the rotor rotates.

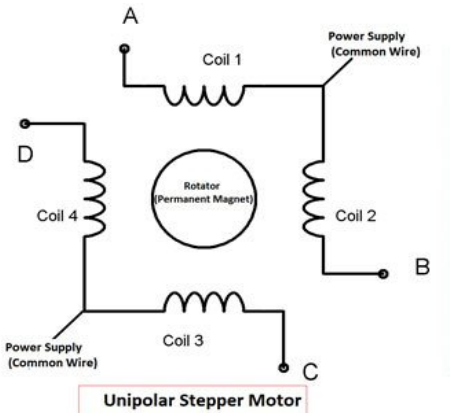


Fig.1: stator windings configuration

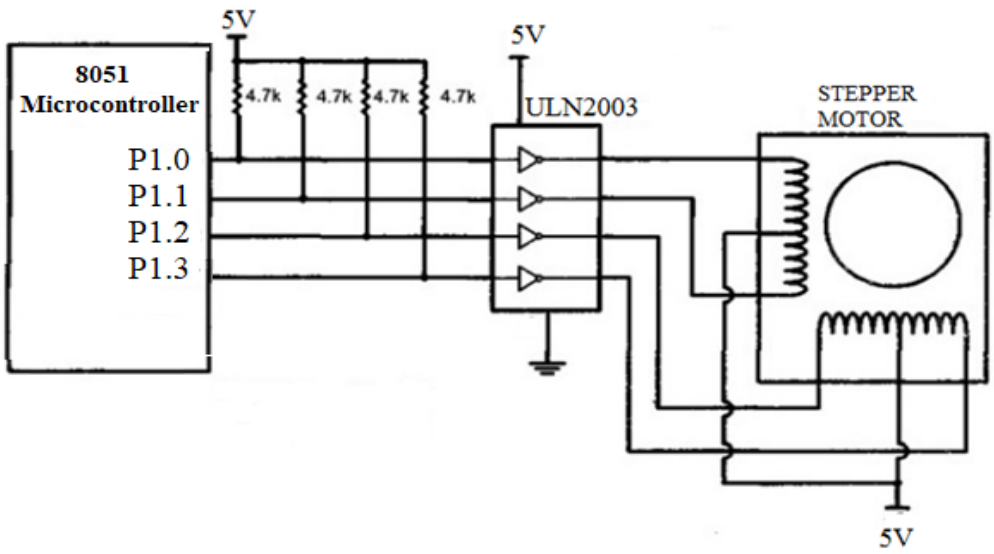


Fig.2: Interfacing of stepper motor with 89x51

There are many sequences for rotation. Each sequence has a different degree of precision. Normal four step sequence has 1.8° angle per step whereas eight step sequence has 0.9° angle per step.

Table 1. shows four stepping sequence whereas Table 2. Shows eight stepping sequence.

Although we can start with any sequence i.e. clockwise or anticlockwise. If we start with the sequence as 1,2,3,4 then motor will rotate in clockwise direction (RRC) and if we start with the sequence as 4,3,2,1 then motor will rotate in anticlockwise direction (RLC).

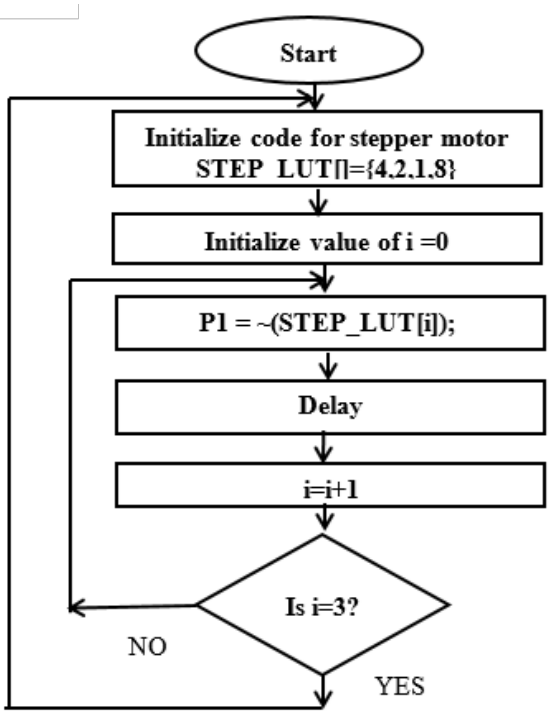
Table 1. Four step sequence (Full stepping sequence)

STEP	WINDING A1	WINDING A2	WINDING A3	WINDING A4
1	1	0	0	1
2	1	1	0	0
3	0	1	1	0
4	0	0	1	1

Table 2. Eight step sequence (Half stepping sequence)

STEP	WINDING A1	WINDING A2	WINDING A3	WINDING A4
1	**1**	**0**	**0**	**1**
2	**1**	**0**	**0**	**0**
3	**1**	**1**	**0**	**0**
4	**0**	**1**	**0**	**0**
5	**0**	**1**	**1**	**0**
6	**0**	**0**	**1**	**0**
7	**0**	**0**	**1**	**1**
8	**0**	**0**	**0**	**1**

Flow Chart for Stepper motor:



Program: -

```
#include<REG51.h>

unsigned int STEP_LUT[]={4,2,1,8};

void Delayms(unsigned int delay)
{
   int i,j;

   for(i=0;i<delay;i++)
   {
      for(j=0;j<50;j++);
   }
}

void main(void)
{
   unsigned int i,delayOne=5;

   while(1)
   {
```

```
        for(i=0;i<=3;i++)
        {
          P2=~(STEP_LUT[i]);
          Delayms(delayOne);
        }
    }
}
```

Procedure to use the Keil software:-

Step 1] Open Keil 5 Software
Step 2] Close old Project
Step 3] Create new Project
Step 4] Save file with extension .c
Step 5] Search

- Select **SST89E564RD**
- Click Ok

Step 6] Right Click on Target-1

- Click on Output
- Click on Create Hex file
- Click Ok

Step 7] Right Click on Source

- Click on Add Existing File
- Add Created File (ex.c)

Step 8] Double click on Added file
Step 9] Type Program

- Save
- Rebuild

Step 10] Right click on my Computer

- Click on Properties
- Click on Device Manager
- Click on Port(ex.port5) *//Remind Port No*.

Step 11] Right click on SSTFLASHFLEX Software

- Click on Run as Administrator
- Click Yes

Step 12] Click on Option

- Check whether Device name and Chip Type is same or Select SST89E/V564RD
- Select COMPORT (ex.port5)
- Click Ok

Step 13] Click RESET button on Kit

Step 14] Click on Dnload/Load

- Click on Browse File
- Select Hex file we created
- Click Ok

Conclusion: -

The stepper motor is successfully interfaced with the 8051 microcontroller. The motor is successfully controlled in the required direction and stepping sequence.

EXPERIMENT NO: 6

TITLE: - INTERFACING DC MOTOR WITH 8051 MICROCONTROLLERS

Aim: To interface a DC motor with 8051 microcontroller using motor driver IC for controlling motor speed and direction in embedded control applications.

Apparatus: 89c51 microcontroller kit, DC motor, L293D motor driver IC, connecting wires.

THEORY:

A DC motor consists mainly of a **field system, armature, commutator and brushes**. The field system produces the magnetic field, while the armature carries current and rotates within the magnetic field. The brushes supply electrical current to the rotating armature through the commutator.

The operation of a DC motor is based on the principle that when a current carrying conductor is placed in a magnetic field, it experiences a mechanical force. This force produces torque and causes the armature to rotate.

The direction of rotation of a DC motor depends on the direction of current flowing through the armature. Therefore, reversing the polarity of the supply across the motor terminals reverses the direction of rotation.

DC motors are widely used in **robotics, electric vehicles, fans, pumps, toys, conveyor systems and automatic control systems** because they provide simple and reliable rotational motion.

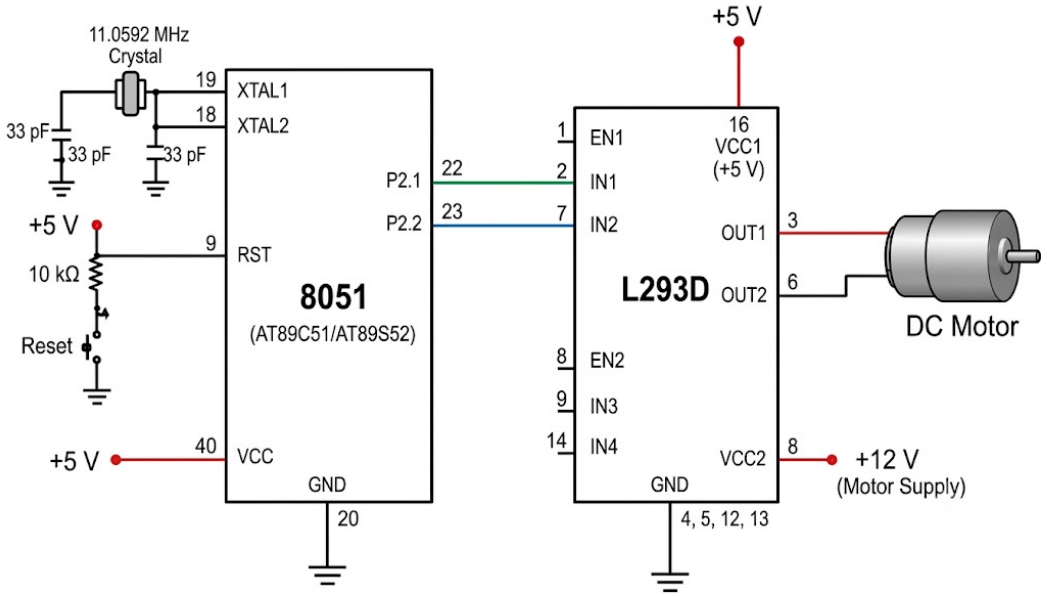


Fig.1: Interfacing of DC motor with 8051 using L293D

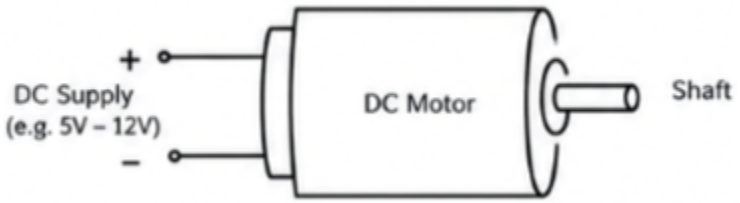


Fig.2: DC Motor

A DC motor has two terminals. By changing the polarity of the voltage applied across these terminals, the direction of rotation of the motor can be reversed. However, the 8051 microcontrollers cannot directly drive the motor because its port pins cannot supply the required motor current. Therefore, a motor driver IC is used between the 8051 microcontroller and the DC motor.

Motor Driver IC:

L293D is a dual H-bridge motor driver IC used to interface a DC motor with the 8051 microcontrollers. It provides the required current to the motor and allows the microcontroller to control the direction of rotation. The input pins of L293D are connected to the port pins of 8051 and the output pins are connected to the DC motor.

In this interfacing, **P2.1 and P2.2** of the 8051 are connected to **IN1 and IN2** of the L293D respectively. The output pins **OUT1 and OUT2** of L293D are connected to the two terminals of the DC motor.

The direction of rotation is controlled by the logic signals applied to IN1 and IN2. When **IN1 = 1 and IN2 = 0**, the motor rotates in one direction. When **IN1 = 0 and IN2 = 1**, the motor rotates in the opposite direction. When both input pins have the same logic level, the motor stops.

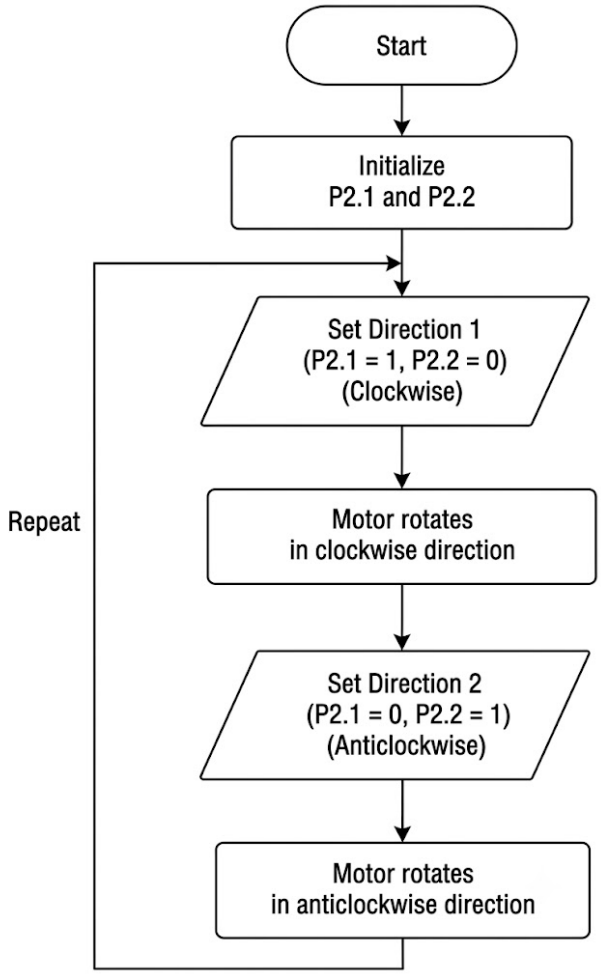
Direction Control Table: -

IN1	IN2	Motor Direction
0	0	Stop
1	0	Clockwise
0	1	Anticlockwise
1	1	Stop

The L293D acts as an interface between the 8051 and the DC motor. The microcontroller provides only the control signals, while the motor driver supplies the required current to the motor. Hence, the motor can be safely controlled without connecting it directly to the 8051 port.

By changing the logic combination of **P2.1 and P2.2**, the direction of the DC motor can be changed from clockwise to anticlockwise and vice versa. This type of direction control is

Flowchart:



Program:

```
#include<REG51.h>
sbit DC1 = P2^1;
sbit DC2 = P2^2;
```

```
void delay(void)
{
   unsigned int i,j;

   for(i=0;i<0xFF; i++)
   {
      for(j=0;j<0xFF;j++);
   }
}

void main(void)
{
   while(1)
   {
      DC1 = 0;
      DC2 = 1;
      delay();

      DC1 = 0;
      DC2 = 0;
      delay();

      DC1 = 1;
      DC2 = 0;
      delay();

      DC1 = 0;
      DC2 = 0;
      delay();
   }
}
```

Procedure to use the Keil software: -

Step 1] Open Keil 5 Software

Step 2] Close old Project

Step 3] Create new Project

Step 4] Save file with extension .c

Step 5] Search

- Select SST**89E564RD**
- Click Ok

Step 6] Right Click on Target-1

- Click on Output
- Click on Create Hex file
- Click Ok

Step 7] Right Click on Source

- Click on Add Existing File
- Add Created File (ex.c)

Step 8] Double click on Added file

Step 9] Type Program

- Save
- Rebuild

Step 10] Right click on my Computer

- Click on Properties
- Click on Device Manager
- Click on Port(ex.port5) *//Remind Port No*.

Step 11] Right click on SSTFLASHFLEX Software

- Click on Run as Administrator
- Click Yes
-

Step 12] Click on Option

- Check whether Device name and Chip Type is same or Select SST89E/V564RD

- Select COMPORT (ex.port5)
- Click Ok

Step 13] Click RESET button on Kit

Step 14] Click on Dnload/Load

- Click on Browse File
- Select Hex file we created
- Click Ok

Conclusion: -

The DC motor is successfully interfaced with the 8051 microcontroller using the L293D driver. Its direction of rotation is successfully controlled by changing the input logic signals.

EXPERIMENT NO: 7

TITLE: - TO INTERFACE AN LED WITH THE RL78 MICROCONTROLLER

Aim: - To interface an LED with the RL78 microcontroller and write a program for LED blinking for understanding digital output port operations and delay generation.

Apparatus: - Personal Computer (PC), RL78 microcontroller kit, IR obstacle detection sensor, LED, connecting wires, Renesas e² studio IDE with CCRL compiler, and Renesas Flash Programmer (RFP) software.

Theory: -

An LED (Light Emitting Diode) is a semiconductor output device that produces light when current flows through it. LEDs are commonly used as indicators in embedded systems to display the operating condition or output status of a circuit.

The RL78 microcontroller provides digital input/output ports which can be configured according to the requirement of the application. A port configured as an output can be used to control external devices such as LEDs. In this experiment, Port 2 is used for digital output operation.

To interface LEDs with the RL78 microcontroller, the LEDs are connected to the selected output port pins through suitable current-limiting resistors. The resistors limit the current flowing through the LEDs and protect both the LEDs and the microcontroller port pins.

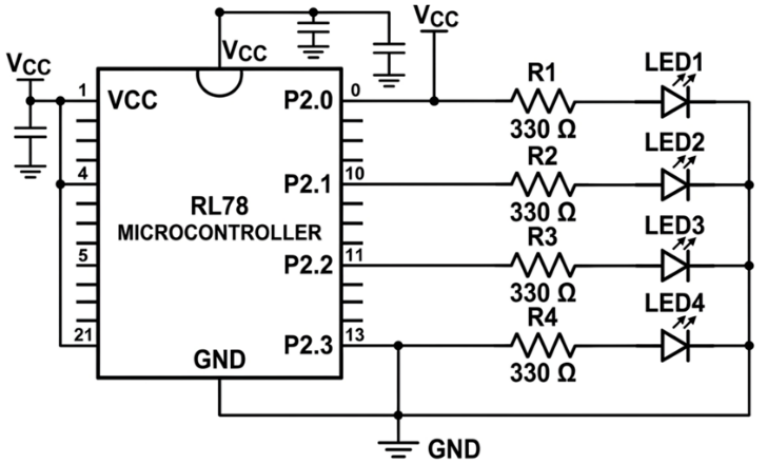


Fig.1: Interfacing of LEDs with RL78 Microcontroller

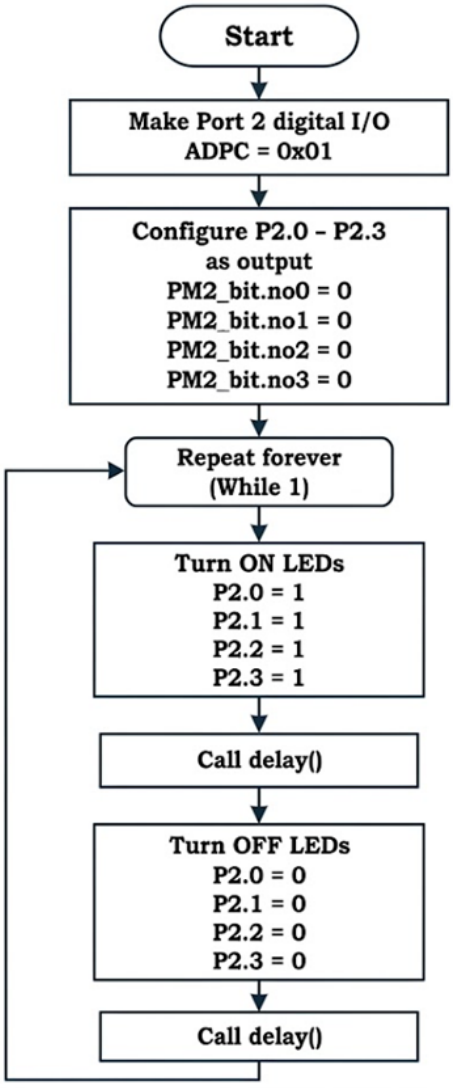
In the given program, P2.0, P2.1, P2.2 and P2.3 are used as output pins to control four LEDs. The required port pins are configured as digital outputs before the LEDs are operated. The program continuously changes the logic state of these pins to turn the LEDs ON and OFF.

When a logic HIGH is applied to the selected output pins, the LEDs are turned ON according to the interfacing connection. When the output pins are changed to logic LOW, the LEDs are turned OFF. Thus, the output state of the microcontroller directly controls the LED indication.

A delay is introduced between the ON and OFF states so that the change in LED state can be observed. The duration of the delay determines the blinking rate of the LEDs. In the program, the delay is generated using nested software loops.

The LED ON and LED OFF operations are placed inside an infinite loop. Therefore, the sequence is continuously repeated and the four LEDs blink together. This experiment demonstrates the use of RL78 digital output ports, port configuration, external LED interfacing and software delay generation.

Flowchart:



Program:

```
#include "iodefine.h"
#define LED_PORT20  P2_bit.no0
#define LED_PORT21  P2_bit.no1
#define LED_PORT22  P2_bit.no2
#define LED_PORT23  P2_bit.no3

void delay(void)
{
   unsigned int i,j;
   for(i=0; i<100; i++)
   {
      for (j=0; j<500; j++);
   }
}

void main(void)
{
   ADPC = 0x01; //make port2 digital IO
   /* Configure P20-P23 as outputs */
   PM2_bit.no0 = 0;
   PM2_bit.no1 = 0;
   PM2_bit.no2 = 0;
   PM2_bit.no3 = 0;

   while (1)
   {
      /* LEDs ON */
      LED_PORT20 = 1;
```

```
        LED_PORT21 = 1;
        LED_PORT22 = 1;
        LED_PORT23 = 1;
        delay ();

        /* LEDs OFF */
        LED_PORT20 = 0;
        LED_PORT21 = 0;
        LED_PORT22 = 0;
        LED_PORT23 = 0;
        delay ();
    }
}
```

Procedure: -

Step 1] Open Renesas e² studio Software

Step 2] Close old Project

Step 3] Create new Project

Step 4] Save file with extension .c

Step 5] Select the required RL78 microcontroller

- Select the required RL78 Device
- Click Ok

Step 6] Configure the Project

- Select the required device settings
- Click Ok

Step 7] Right Click on Source

- Click on Add Existing File
- Add Created File (ex.c)

Step 8] Double click on Added file

Step 9] Type Program

- Save
- Build Project

Step 10] Connect the RL78 Kit to the Computer

- Connect the supported programming/debugging interface
- Check the connected device

Step 11] Open Renesas Flash Programmer (RFP) Software

- Click on Run as Administrator
- Click Yes

Step 12] Click on Connect/Option

- Check whether Device name and Device Type is same or select the required RL78 device
- Select the appropriate programming interface
- Click Ok

Step 13] Click RESET button on Kit

Step 14] Click on Program/Start

- Select the program file generated by the project
- Click Ok

Conclusion: -

The LEDs are successfully interfaced with the RL78 microcontroller. LED blinking is successfully observed by controlling the digital output ports with a suitable time delay.

EXPERIMENT NO: 8

TITLE: - IR OBSTACLE DETECTION USING RL78 MICROCONTROLLER

Aim: To interface an IR obstacle detection sensor with the RL78 microcontroller for detecting nearby objects and controlling indication devices based on sensor output.

Apparatus: Personal Computer (PC), RL78 microcontroller kit, IR obstacle detection sensor, LED, connecting wires, Renesas e² studio IDE with CCRL compiler, and Renesas Flash Programmer (RFP) software.

Theory:

An IR (Infrared) obstacle detection sensor is an electronic sensing device used to detect the presence of an object without physical contact. It is commonly used in embedded systems for object detection, automation, robotics, and indication applications.

The IR obstacle detection sensor generally consists of an infrared transmitter and an infrared receiver. The transmitter emits infrared radiation, and when an object is present in front of the sensor, the emitted infrared radiation is reflected back towards the receiver. The receiver detects the reflected radiation and produces a corresponding electrical output signal.

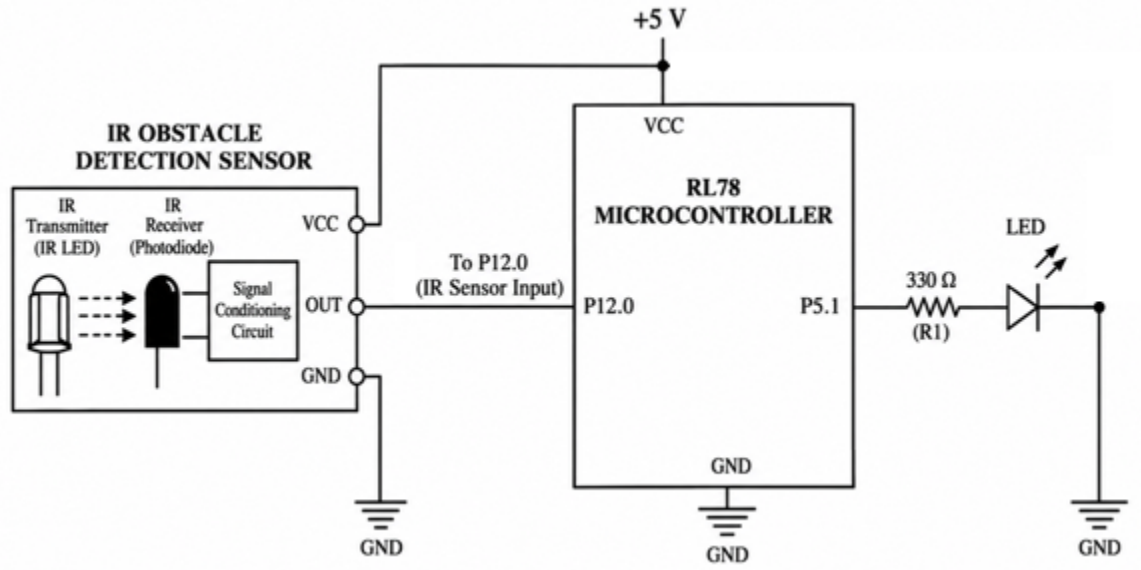


Fig.1: Interfacing of IR Obstacle Detection Sensor with RL78 Microcontroller

In this experiment, the IR obstacle detection sensor is interfaced with the RL78 microcontroller using **P12.0** as the digital input pin. The P12.0 pin is configured as an input and as a digital port so that the microcontroller can continuously read the output of the sensor.

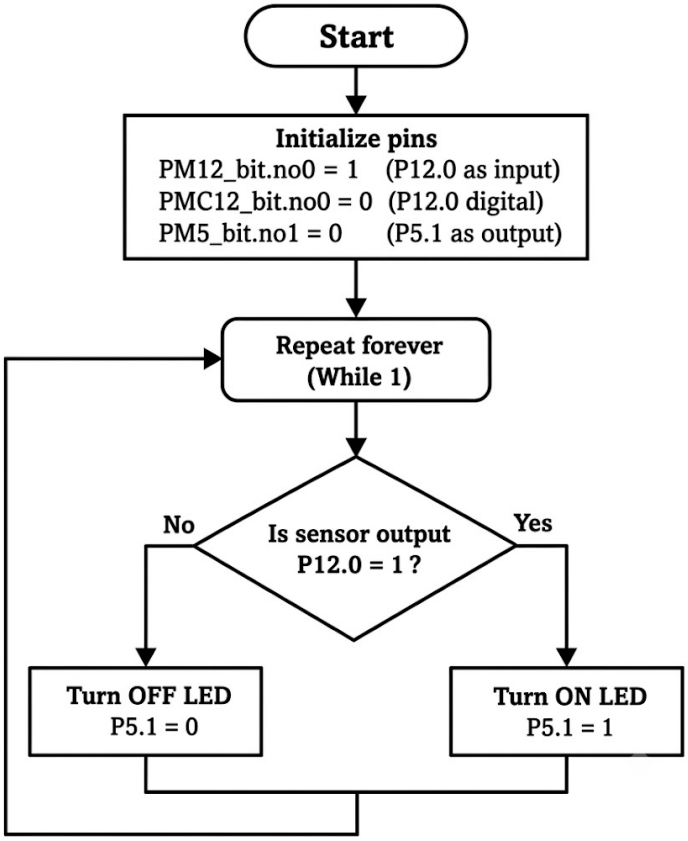
An LED is connected to **P5.1** and is used as the indication device. The P5.1 pin is configured as an output pin. The microcontroller checks the logic state present at P12.0 continuously inside the `while (1)` loop.

According to the given program, when the sensor output at **P12.0 is HIGH**, the microcontroller sets **P5.1 HIGH**, causing the LED to turn ON. When the sensor output is **LOW**, P5.1 is set LOW and the LED turns OFF.

This continuous checking allows the RL78 microcontroller to respond to the sensor output whenever the detected condition changes. The LED therefore provides a simple visual indication corresponding to the output of the IR obstacle sensor.

The experiment demonstrates digital input interfacing, digital output control, and sensor-based operation using the RL78 microcontroller. It shows how an external sensor can provide an input signal to the microcontroller and how that signal can be used to control an output indication device. The RL78 practical sheet specifically includes IR obstacle sensor interfacing as one of its experiments.

Flow Chart: -



Program: -

```
#include "iodefine.h"

void main(void)
{
   PM12_bit.no0 = 1;       //set as input
   PMC12_bit.no0 = 0;      //set as digital
   PM5_bit.no1 = 0;        //LED as output pin

   while (1)
   {
      if (P12_bit.no0 == 1)
      {
         P5_bit.no1 = 1;
      }
      else
      {
         P5_bit.no1 = 0;
      }
   }
}
```

Procedure to use the Renesas e² studio: -

Step 1] Open Renesas e² studio Software

Step 2] Close old Project

Step 3] Create new Project

Step 4] Save file with extension `.c`

Step 5] Select the required RL78 microcontroller

• Select the required RL78 Device

• Click Ok

Step 6] Configure the Project

• Select the required device settings

• Click Ok

Step 7] Right Click on Source

• Click on Add Existing File

• Add Created File (ex.c)

Step 8] Double click on Added file

Step 9] Type Program

• Save

• Build Project

Step 10] Connect the RL78 Kit to the Computer

• Connect the supported programming/debugging interface

• Check the connected device

Step 11] Open Renesas Flash Programmer (RFP) Software

• Click on Run as Administrator

• Click Yes

Step 12] Click on Connect/Option

• Check whether Device name and Device Type is same or select the required RL78 device

• Select the appropriate programming interface

• Click Ok

Step 13] Click RESET button on Kit

Step 14] Click Program/Start

• Select the program file generated by the project

• Click Ok

Conclusion: -

The IR obstacle detection sensor is successfully interfaced with the RL78 microcontroller. The sensor output is monitored and the LED is controlled accordingly to indicate obstacle detection.

EXPERIMENT NO: 9

TITLE: 16X2 LCD INTERFACING USING RL78 MICROCONTROLLER

Aim: To interface a 16×2 LCD display module with the RL78 microcontroller and display alphanumeric characters, messages, and system status information.

Apparatus: Personal Computer (PC), RL78 microcontroller kit, 16×2 LCD display module, connecting wires, Renesas e² studio IDE with CCRL compiler, and Renesas Flash Programmer (RFP) software.

Theory:

A 16×2 LCD (Liquid Crystal Display) is a widely used output device in embedded systems for displaying alphanumeric characters, messages, and system status information. It consists of two rows, with each row capable of displaying 16 characters. LCD displays are commonly used to provide a visual interface between the microcontroller and the user.

The LCD consists of data pins and control pins. The **RS (Register Select)** pin is used to select the instruction register or the data register. When RS is `0`, the instruction register is selected, and when RS is `1`, the data register is selected. The **R/W (Read/Write)** pin is used to select the read or write operation. The **Enable (E)** pin is used to latch the command or data supplied to the LCD.

The LCD can operate in either **4-bit mode or 8-bit mode**. In 4-bit mode, an 8-bit command or character is transferred in two 4-bit parts, while in 8-bit mode all eight data bits are transferred together. In the given RL78 program, **Port 1 (P1)** is used for LCD data transfer. The required control pins are configured as output pins according to the program.

Before displaying a message, the LCD is initialized using suitable commands. The `lcd_init()` function performs the initialization by sending commands such as `0x38`, `0x0C`, `0x01`, and `0x06`.

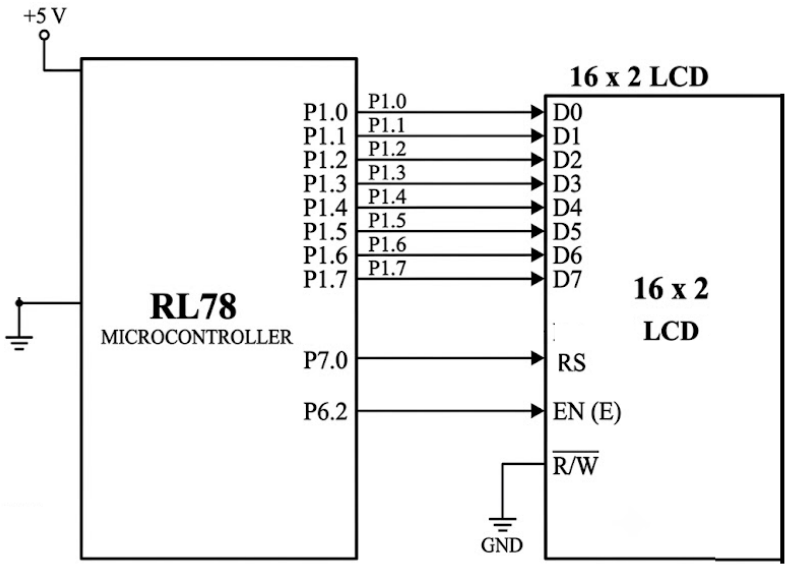


Fig. 1: 16×2 LCD Interfacing with RL78 Microcontroller

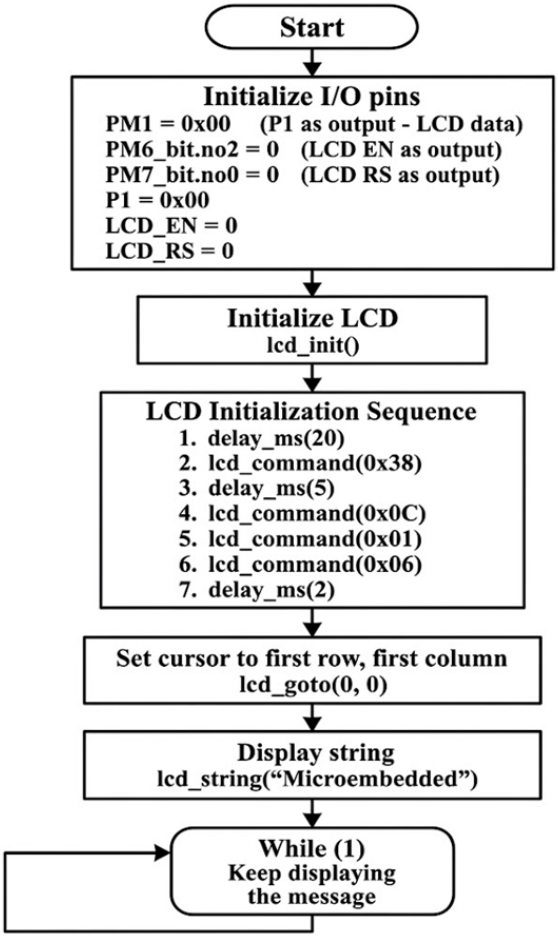
These commands configure the LCD, switch ON the display, clear the display, and set the cursor movement operation.

The `lcd_command()` function is used to send commands to the LCD, whereas the `lcd_data()` function is used to send character data. The `lcd_enable()` function generates the enable signal required during command and data transfer. Delay functions are used between operations to provide sufficient time for the LCD to process the received information.

The `lcd_goto()` function is used to select the required row and column position of the LCD. The `lcd_string()` function sends a complete string character by character. In the main program, the LCD is initialized and the cursor is positioned at the first row and first column. The message **"Microembedded"** is then displayed on the LCD.

The program remains in an infinite loop after displaying the message, so the displayed information remains on the LCD. Thus, this experiment demonstrates LCD initialization, command transmission, character transmission, cursor positioning, and message display using the RL78 microcontroller.

Flow Chart: -



Connection of RL 78 and LCD

EN P62
RS P70
R/W GND
LCD VSS GND
LCD VDD +5V
LCD VO Contrast potentiometer
LCD data bus
P10-P17 correspond directly to D0-D7.
Therefore, the complete Port 1 can be used directly.

Program: -

```
#include "iodefine.h"
void main(void);
/* Delay */
static void delay_us(unsigned int us)
{
   volatile unsigned int i;
   for (i = 0; i < us; i++)
   {
```

```
        __nop();
        __nop();
        __nop();
        __nop();
    }
}

static void delay_ms(unsigned int ms)
{
    volatile unsigned int i;

    for (i = 0; i < ms; i++)
    {
        delay_us(1000);
    }
}

/* LCD control signals */
/* RS = P70 */
#define LCD_RS      P7_bit.no0
/* EN = P62 */
#define LCD_EN      P6_bit.no2

#define LCD_DATA    P1
/* LCD Enable pulse */
static void lcd_enable(void)
{
    LCD_EN = 1;
    /* Enable pulse width */
    delay_us(10);
    LCD_EN = 0;
    /* Allow LCD to process the data */
    delay_us(50);
}

/* Send command to LCD */
static void lcd_command(unsigned char command)
{
    /* RS = 0 -> command */
    LCD_RS = 0;
    /* Put command on D0-D7 */
    LCD_DATA = command;
    /* Enable pulse */
    lcd_enable();

    if ((command == 0x01) || (command == 0x02))
    {
        delay_ms(2);
```

```
    }
}

/* Send data/character to LCD */
static void lcd_data(unsigned char data)
{
    /* RS = 1 -> data */
    LCD_RS = 1;
    /* Put character on D0-D7 */
    LCD_DATA = data;
    /* Enable pulse */
    lcd_enable();
}

/* LCD initialization */
static void lcd_init(void)
{
    /* Wait for LCD power-up */
    delay_ms(20);
    /* Function Set * 8-bit mode,  * 2 lines , * 5x8 font  */
    lcd_command(0x38);
    delay_ms(5);
    /* Display ON, cursor OFF, blink OFF */
    lcd_command(0x0C);
    /* Clear display */
    lcd_command(0x01);
    /* Entry mode: increment cursor */
    lcd_command(0x06);
    delay_ms(2);
}

/* Set LCD cursor position * row = 0 -> first line, * row = 1 -> second line, * column = 0 to 15 */

static void lcd_goto(unsigned char row, unsigned char column)
{
    unsigned char address;
    if (row == 0)
    {
        address = 0x80 + column;
    }
    else
    {
        address = 0xC0 + column;
    }

    lcd_command(address);
}
```

```
/* Display string */
static void lcd_string(const char *str)
{
   while (*str != '\0')
   {
      lcd_data((unsigned char)*str);
      str++;
   }
}

/* MAIN */
void main(void)
{
   /* PORT CONFIGURATION */
   /* P10-P17 = LCD D0-D7, Configure Port 1 as OUTPUT*/
   PM1 = 0x00;
   /* P62 = LCD Enable, * Configure as OUTPUT */
   PM6_bit.no2 = 0;
   /* P70 = LCD RS, * Configure as OUTPUT */
   PM7_bit.no0 = 0;

  /* INITIAL OUTPUT STATES */
   P1 = 0x00;
   LCD_EN = 0;
   LCD_RS = 0;

   /* INITIALIZE LCD */
   lcd_init();

  /* DISPLAY MESSAGE */
   lcd_goto(0, 0);
   lcd_string("Microembedded");

   /* MAIN LOOP */
   while (1)
   {
      /* Keep displaying the message */
   }
}
```

Procedure to use the Renesas e² Studio: -

Step 1] Open Renesas e² studio Software

Step 2] Close old Project

Step 3] Create new Project

Step 4] Save file with extension `.c`

Step 5] Select the required RL78 microcontroller

- Select the required RL78 Device
- Click Ok

Step 6] Configure the Project

- Select the required device settings
- Click Ok

Step 7] Right Click on Source

- Click on Add Existing File
- Add Created File (ex.c)

Step 8] Double click on Added file

Step 9] Type Program

- Save
- Build Project

Step 10] Connect the RL78 Kit to the Computer

- Connect the supported programming/debugging interface
- Check the connected device

Step 11] Open Renesas Flash Programmer (RFP) Software

- Click on Run as Administrator
- Click Yes

Step 12] Click on Connect/Option

- Check whether Device name and Device Type is same or select the required RL78 device
- Select the appropriate programming interface
- Click Ok

Step 13] Click RESET button on Kit

Step 14] Click Program/Start

- Select the program file generated by the project
- Click Ok

Conclusion: -

The 16×2 LCD display is successfully interfaced with the RL78 microcontroller by properly configuring the control and data lines. The message **"Microembedded"** is successfully displayed, confirming proper hardware and software integration.